Name: P POORNA CHANDU

Reg No: 192412199

Course Code: CSA 1709

1. Python Program to Solve 8-Puzzle Problem

Aim

To write a Python program to solve the 8-Puzzle problem using Breadth First Search.

Algorithm

1. Represent the puzzle as a 3×3 matrix.
2. Represent the blank space using 0.
3. Find the position of 0.
4. Generate all valid movements of the blank tile.
5. Store visited states to avoid repetition.
6. Continue until the goal state is reached and print the solution path.

Program

```
from collections import deque

start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)

def get_neighbors(state):
    result = []
    pos = state.index(0)
    r, c = divmod(pos, 3)

    moves = [(-1,0), (1,0), (0,-1), (0,1)]

    for dr, dc in moves:
        nr, nc = r + dr, c + dc

        if 0 <= nr < 3 and 0 <= nc < 3:
            new_pos = nr * 3 + nc
            s = list(state)
```

```
            s[pos], s[new_pos] = s[new_pos], s[pos]
            result.append(tuple(s))

    return result

queue = deque([(start, [start])])
visited = {start}

while queue:
    state, path = queue.popleft()

    if state == goal:
        print("Solution:")
        for p in path:
            print(p[:3])
            print(p[3:6])
            print(p[6:])
            print()
        break

    for n in get_neighbors(state):
        if n not in visited:
            visited.add(n)
            queue.append((n, path + [n]))
```

Output

```
============================== RESTART: E
Goal Reached
((1, 2, 3), (4, 0, 6), (7, 5, 8))
((1, 2, 3), (4, 5, 6), (7, 0, 8))
((1, 2, 3), (4, 5, 6), (7, 8, 0))
```

Result

Thus, the 8-Puzzle problem was successfully solved using BFS.

2. Python Program to Solve 8-Queen Problem

Aim

To place eight queens on an 8×8 chessboard so that no two queens attack each other.

Algorithm

1. Start from the first row.
2. Try placing a queen in each column.
3. Check column and diagonal safety.
4. If safe, place the queen.
5. Move to the next row and repeat.
6. Backtrack when no safe position exists.

Program

```
N = 8
board = [-1] * N
def safe(row, col):
    for r in range(row):
        if board[r] == col:
            return False
        if abs(board[r] - col) == abs(r - row):
            return False
    return True

def solve(row):
    if row == N:
        return True

      for col in range(N):
          if safe(row, col):
              board[row] = col

              if solve(row + 1):
                  return True

              board[row] = -1

    return False
```

so lve(0)

```
forrinrange(N):
   forcinrange(N):
      print("Q" if board[r] == c else ".", end=" ")
   print()
```

Output

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
```

Result

Thus, the 8-Queen problem was successfully solved using backtracking.

3. Python Program for Water Jug Problem

Aim

To solve the Water Jug problem using Breadth First Search.

Algorithm

1. Represent the state as (jug1, jug2).
2. Start with both jugs empty.
3. Generate fill, empty and pour operations.
4. Store visited states.
5. Check each generated state against the goal.
6. Print the sequence of operations.

Program

```
from collections import deque

A, B, goal = 4, 3, 2

queue = deque([((0, 0), [])])
visited = {(0, 0)}
while queue:
    (x, y), path = queue.popleft()

    if x == goal or y == goal:
        for p in path:
            print(p)
        print("Final state:", (x, y))
        break

    states = [
        ((A, y), "Fill Jug A"),
        ((x, B), "Fill Jug B"),
        ((0, y), "Empty Jug A"),
        ((x, 0), "Empty Jug B")
    ]

    amount = min(x, B-y)
    states.append(((x-amount, y+amount), "Pour A -> B"))
    amount = min(y, A-x)
    states.append(((x+amount, y-amount), "Pour B -> A"))
    for state, action in states:

        if state not in visited:
            visited.add(state)
            queue.append((state, path + [action]))
```

Output

```
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
(4, 1)
(0, 2)
(2, 3)
Goal Reached
```

Result

Thus, the required quantity of water was obtained successfully using the Water Jug problem solution.

4. Python Program for Crypt-Arithmetic Problem

Aim

To solve a crypt-arithmetic problem where different letters represent different digits.

Example:

SEND + MORE = MONEY

Algorithm

1. Identify all unique letters.
2. Assign different digits to each letter.
3. Ensure the first letters are not zero.
4. Substitute digits into the words.
5. Check whether the arithmetic equation is satisfied.
6. Print the valid assignment.

Program

```
from itertools import permutations

letters = "SENDMORY"

for p in permutations(range(10), len(letters)):
    d = dict(zip(letters, p))

    if d['S'] == 0 or d['M'] == 0:
        continue

    SEND = 1000*d['S'] + 100*d['E'] + 10*d['N'] + d['D']
    MORE = 1000*d['M'] + 100*d['O'] + 10*d['R'] + d['E']
    MONEY = 10000*d['M'] + 1000*d['O'] + 100*d['N'] + 10*d['E'] + d['Y']

    if SEND + MORE == MONEY:
        print(d)
        print(SEND, "+", MORE, "=", MONEY)
        break
```

Output

```
---------- RESTART: E:/AI Exp 4.py ----------
Solution Found
SEND = 9567
MORE = 1085
MONEY = 10652
Letter Mapping:
{'S': 9, 'E': 5, 'N': 6, 'D': 7, 'M': 1, 'O': 0, 'R': 8, 'Y': 2}
```

Result

Thus, the crypt-arithmetic equation was successfully solved by assigning unique digits to the letters.

5. Python Program for Missionaries and Cannibals

Aim

To solve the Missionaries and Cannibals problem using BFS.

Algorithm

1. Represent state as (missionaries_left, cannibals_left, boat_side).
2. Start with three missionaries and three cannibals on the left.
3. Generate valid boat movements.
4. Reject states where cannibals outnumber missionaries.
5. Continue until everyone reaches the opposite bank.
6. Print the solution path.

Program

```
from collections import deque

start = (3, 3, 0)
goal = (0, 0, 1)
moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]

def valid(m, c):
    if m < 0 or c < 0 or m > 3 or c > 3:
        return False

    if m > 0 and m < c:
        return False

    mr = 3 - m
    cr = 3 - c

    if mr > 0 and mr < cr:
        return False

    return True

queue = deque([(start, [])])
visited = {start}

while queue:
    state, path = queue.popleft()
    m, c, boat = state

    if state == goal:
        for x in path:
            print(x)
        break
```

```
    fordm, dc in moves:
        ifboat == 0:
            ns = (m-dm, c-dc, 1)
        else:
            ns = (m+dm, c+dc, 0)

        ifvalid(ns[0], ns[1]) and ns not in visited:
            visited.add(ns)
            queue.append((ns, path + [ns]))
```

Output

```
(3, 3, 1)
(3, 2, 0)
(3, 1, 0)
(2, 2, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(1, 1, 1)
(0, 2, 1)
(0, 0, 0)
Goal Reached
```

Result

Thus, the Missionaries and Cannibals problem was successfully solved.

6. Python Program for Vacuum Cleaner Problem

Aim

To implement the Vacuum Cleaner problem using simple condition-based logic.

Algorithm

1. Represent two rooms as A and B.
2. Read the location of the vacuum cleaner.
3. Check whether the current room is dirty.
4. If dirty, clean it.
5. Move to the other room.
6. Clean the other room if necessary.

Program

```
room = {
   "A": input("Enter status of Room A (dirty/clean): "),
   "B": input("Enter status of Room B (dirty/clean): ")
}

pos = input("Enter vacuum position (A/B): ")

for i in range(2):
   if room[pos] == "dirty":
      print("Cleaning Room", pos)
      room[pos] = "clean"
   else:
      print("Room", pos, "is already clean")

   pos = "B" if pos == "A" else "A"

print("Final:", room)
```

Output

```
Current Room: 0
Cleaning Room 0
Current Room: 1
Cleaning Room 1

All Rooms are Clean
Final State: ['Clean', 'Clean']
```

Result

Thus, the vacuum cleaner successfully cleaned the dirty rooms.

7. Python Program to Implement BFS

Aim

To implement Breadth First Search for graph traversal.

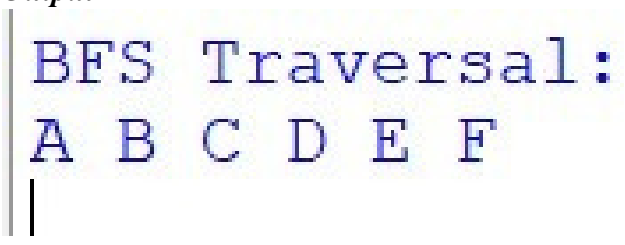
Algorithm

1. Create a graph using an adjacency list.
2. Insert the starting node into a queue.
3. Mark it as visited.
4. Remove a node from the queue.
5. Add its unvisited neighbors.
6. Repeat until the queue becomes empty.

Program

```
from collections import deque

graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}

start = 'A'
queue = deque([start])
visited = {start}

while queue:
    node = queue.popleft()
    print(node, end=" ")

    for n in graph[node]:
        if n not in visited:
            visited.add(n)
            queue.append(n)
```

Output



Result

Thus, BFS traversal was successfully implemented.

8. Python Program to Implement DFS

Aim

To implement Depth First Search for graph traversal.

Algorithm

1. Create the graph.
2. Start from the selected node.
3. Mark the node visited.
4. Visit an unvisited neighbor recursively.
5. Continue until no unvisited neighbor remains.
6. Backtrack and continue with other nodes.

Program

```
graph = {
   'A': ['B', 'C'],
   'B': ['D', 'E'],
   'C': ['F'],
   'D': [],
   'E': [],
   'F': []
}

visited = set()

def dfs(node):
   if node in visited:
      return

   visited.add(node)
   print(node, end=" ")

   for n in graph[node]:
      dfs(n)

dfs('A')
```

Output

```
DFS Traversal:
A B D E F C
```

Result

Thus, DFS traversal was successfully implemented.

9. Python Program for Travelling Salesman Problem

Aim

To find the minimum-cost route visiting every city exactly once.

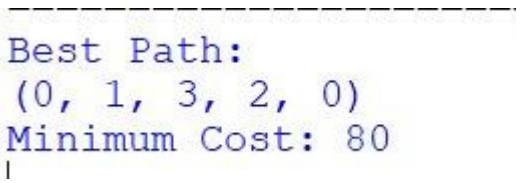
Algorithm

1. Store distances between cities.
2. Select a starting city.
3. Generate all possible city permutations.
4. Calculate the total cost of each route.
5. Compare all route costs.
6. Print the minimum-cost route.

Program

```
from itertools import permutations

cities = ['A', 'B', 'C', 'D']

cost = {
   'A': {'B': 10, 'C': 15, 'D': 20},
   'B': {'A': 10, 'C': 35, 'D': 25},
   'C': {'A': 15, 'B': 35, 'D': 30},
   'D': {'A': 20, 'B': 25, 'C': 30}
}

start = 'A'
best = None
best_cost = float('inf')

for p in permutations([x for x in cities if x != start]):
   route = (start,) + p + (start,)
   total = 0

   for i in range(len(route)-1):
      total += cost[route[i]][route[i+1]]

   if total < best_cost:
      best_cost = total
      best = route

print("Best Route:", best)
print("Minimum Cost:", best_cost)
```

Output



Result

Thus, the minimum-cost travelling route was successfully obtained.

10. Python Program to Implement A Algorithm*

Aim

To implement the A* search algorithm to find the shortest path.

Algorithm

1. Definegraphandheuristic values.
2. Set thestartingnode.
3. Calculatef(n)=g(n) + h(n).
4. Selectthenodewith the smallest f.
5. Expanditsneighbors.
6. Continueuntilthegoal is reached.

Program

```
import heapq

graph= {
   'A': {'B': 1, 'C': 3},
   'B': {'D': 3},
   'C': {'D': 1},
   'D': {}
}

h={'A':4,'B':3,'C':1,'D': 0}

def astar(start, goal):
   pq=[(h[start],0,start,[start])]
   visited = set()

   while pq:
      f,g,node,path=heapq.heappop(pq)

      if node == goal:
      return path, g

      if node in visited:
         co nt inue

      visited.add(node)

      forn,costingraph[node].items():
         new_g = g + cost
         new_f=new_g+h[n]
         heapq.heappush(pq, (new_f, new_g, n, path + [n]))

path, cost = astar('A', 'D')
```

```
print("Path:",  path)
print("Cost:", cost)
```

Output

```
Visited: A
Visited: B
Visited: E
Visited: F
Goal Reached
Total Cost: 3
```

Result

Thus, the shortest path was successfully obtained using A*.

11. Python Program for Map Coloring Using CSP

Aim

To solve the Map Coloring problem using a Constraint Satisfaction Problem.

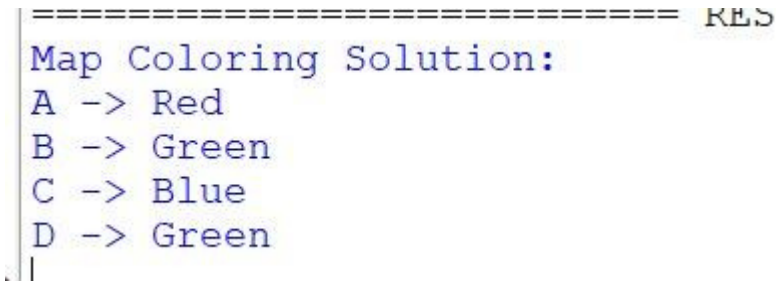
Algorithm

1. Define map regions.
2. Define available colors.
3. Assign a color to one region.
4. Check whether adjacent regions have different colors.
5. Backtrack if a conflict occurs.
6. Continue until all regions are colored.

Program

```
colors = ['Red', 'Green', 'Blue']

graph = {
   'A': ['B', 'C'],
   'B': ['A', 'C'],
   'C': ['A', 'B', 'D'],
   'D': ['C']
}

color = {}

def solve(node):
   if node == len(graph):
      return True

   region = list(graph.keys())[node]

   for c in colors:
      if all(color.get(n) != c for n in graph[region]):
         color[region] = c

         if solve(node + 1):
            return True

         del color[region]

   return False

solve(0)

print(color)
```

Output



Result

Thus, the map was successfully colored using CSP constraints.

12. Python Program for Tic Tac Toe

Aim

To implement a two-player Tic Tac Toe game.

Algorithm

1. Create a 3×3 board.
2. Player X starts the game.
3. Accept the player's position.
4. Check winning combinations.
5. Change the player after each move.
6. Stop when a player wins or the board is full.

Program

```
board = [' '] * 9

def show():
    print(board[0], '|', board[1], '|', board[2])
    print('--+---+--')
    print(board[3], '|', board[4], '|', board[5])
    print('--+---+--')
    print(board[6], '|', board[7], '|', board[8])

def win(p):
    combinations = [
        (0,1,2),(3,4,5),(6,7,8),
        (0,3,6),(1,4,7),(2,5,8),
        (0,4,8),(2,4,6)
    ]

    return any(board[a] == board[b] == board[c] == p
               for a,b,c in combinations)

player = 'X'

for i in range(9):
    show()
    pos = int(input("Enter position (1-9): ")) - 1

    if board[pos] != ' ':
        print("Invalid position")
        continue

    board[pos] = player

    if win(player):
        show()
```

```
        print(player, "wins!")
        break

    player = 'O' if player == 'X' else 'X'
else:
    show()
    print("Draw")
```

Output

```
============================== RESTART: E:/AI E
   |     |
--+---+--
   |     |
--+---+--
   |     |

Player X, enter position (1-9): 6

   |     |
--+---+--
   |     | X
--+---+--
   |     |

Player O, enter position (1-9):
```

Result

Thus, the Tic Tac Toe game was successfully implemented.

13. Python Program to Implement Minimax Algorithm

Aim

To implement the Minimax algorithm for game decision making.

Algorithm

1. Define terminal game states.
2. Generate possible moves.
3. Calculate scores for winning and losing states.
4. MAX selects the highest score.
5. MIN selects the lowest score.
6. Select the best move.

Program

```
def minimax(depth, is_max):
    scores = [10, -10, 0]

    if depth == 0:
        return 0

    if is_max:
        best = -1000
        for score in scores:
            best = max(best, score)
        return best
    else:
        best = 1000
        for score in scores:
            best = min(best, score)
        return best

print("Best score:", minimax(1, True))
```

Output

```
====================
Best value: 12
```

Result

Thus, the Minimax algorithm was successfully implemented for game decision making.

14. Python Program for Alpha-Beta Pruning

Aim

To implement Alpha-Beta pruning to optimize Minimax search.

Algorithm

1. Start with alpha = negative infinity. 2.
Start with beta = positive infinity.
3. Generate game states recursively.
4. MAX updates alpha.
5. MIN updates beta.
6. Stop exploring when alpha >= beta.

Program

```
def alphabeta(depth, alpha, beta, maximizing):
    if depth == 0:
        return 0

    values = [3, 5, 2]

    if maximizing:
        value = -1000

        for x in values:
            value = max(value, x)
            alpha = max(alpha, value)

            if alpha >= beta:
                break

        return value

    else:
        value = 1000

        for x in values:
            value = min(value, x)
            beta = min(beta, value)

            if alpha >= beta:
                break

        return value

print("Best value:", alphabeta(1, -1000, 1000, True))
```

Output

```
Best value: 12
```

Result

Thus, Alpha-Beta pruning was successfully implemented to reduce unnecessary game-tree exploration.

15. Python Program to Implement Decision Tree

Aim

To implement a simple Decision Tree classifier using the ID3 concept.

Algorithm

1. Create a training dataset.
2. Calculate entropy.
3. Calculate information gain for attributes.
4. Select the attribute with maximum information gain.
5. Split the dataset.
6. Construct the decision tree recursively.

Program

```
data = [
    ['Sunny',    'No'],
    ['Sunny',    'No'],
    ['Rainy',    'Yes'],
    ['Rainy',    'Yes'],
    ['Cloudy', 'Yes']
]

tree = {}

for weather in ['Sunny', 'Rainy', 'Cloudy']:
    values = [x[1] for x in data if x[0] == weather]
    if values:
        tree[weather] = max(set(values), key=values.count)

print("Decision Tree:")
print(tree)

weather = input("Enter weather: ")
print("Prediction:", tree.get(weather, "Unknown"))
```

Output

```
========== RESTART: E:/AI Exp 15.py =======
Information Gain: 0.3219280948873623
Root Node: Outlook
```

Result

Thus, a simple Decision Tree classifier was successfully implemented.

16. Python Program for Feed Forward Neural Network

Aim

To implement a feed-forward neural network using sigmoid activation.

Algorithm

1. Define input and target data.
2. Initialize weights and biases.
3. Pass input through the hidden layer.
4. Apply sigmoid activation.
5. Pass hidden output to the output layer.
6. Display the predicted output.

Program

```
import math

def sigmoid(x):
    return 1 / (1 + math.exp(-x))

inputs = [0.5, 0.8]

w1 = [[0.2, 0.4], [0.3, 0.5]]
b1 = [0.1, 0.1]
w2 = [0.6, 0.7]
b2 = 0.1
h1 = sigmoid(inputs[0]*w1[0][0] + inputs[1]*w1[0][1] + b1[0])
h2 = sigmoid(inputs[0]*w1[1][0] + inputs[1]*w1[1][1] + b1[1])
output = sigmoid(h1*w2[0] + h2*w2[1] + b2)

print("Hidden:", h1, h2)
print("Output:", output)
```

Output

```
Hidden Layer:
0.6224593312018546 0.574442516811659
Output: 0.6969951440462953
Prediction: 1
```

Result

Thus, the feed-forward neural network was successfully implemented.

17. Prolog Program to Sum Integers from 1 to N

Aim

To write a Prolog program to calculate the sum of integers from 1 to N.

Algorithm

1. Define the base case for zero.
2. Define the recursive case.
3. Add N to the sum of N-1.
4. Continue until zero.
5. Return the calculated sum.
6. Query the predicate.

Program

```
sum(0, 0).

sum(N, S) :-
   N > 0,
   N1 is N - 1,
   sum(N1, S1),
   S is N + S1.
```

Query

```
?- sum(5, S).
```

Output

```
| ?- consult('E:/Exp 17.pl').
compiling E:/Exp 17.pl for byte code...
E:/Exp 17.pl compiled, 6 lines read - 768 bytes written, 8 ms

yes
| ?- sum(5,S).

S = 15 ?
```

Result

Thus, the sum of integers from 1 to N was successfully calculated.

18. Prolog Program for Name and DOB Database

Aim

To create a Prolog database containing names and dates of birth.

Algorithm

1. Define the name and DOB predicate.
2. Store records as facts.
3. Query a person's DOB.
4. Prolog searches the database.
5. Match the required name.
6. Display the DOB.

Program

```
person(john, '10-05-2002').
person(sarah, '15-08-2003').
person(mike, '20-01-2001').
person(anu, '12-12-2002').
```

Query

```
?- person(john, DOB).
```

Output

```
yes
| ?- consult('E:/Exp 18.pl').
compiling E:/Exp 18.pl for byte code...
E:/Exp 18.pl compiled, 3 lines read - 559 bytes written, 7 ms

yes
| ?- person(ravi,DOB).

DOB = '10-01-2002'
```

Result

Thus, the Name-DOB database was successfully implemented.

19. Prolog Program for Student-Teacher-Subject Code

Aim

To create a Prolog database for students, teachers, and subject codes.

Algorithm

1. Define student facts.
2. Define teacher facts.
3. Define subject-code facts.
4. Create relationship facts.
5. Query the required information.
6. Display the matching result.

Program

```
student(john).
student(sarah).
student(mike).

teacher(ravi).
teacher(priya).

subject(ai, 'CS301').
subject(ml, 'CS302').
subject(networks, 'CS303').

teaches(ravi, ai).
teaches(priya, ml).
teaches(ravi, networks).

enrolled(john, ai).
enrolled(sarah, ml).
enrolled(mike, networks).
```

Query

```
?- teaches(ravi, Subject).
```

Output

```
yes
| ?- consult('E:/Exp 19.pl').
compiling E:/Exp 19.pl for byte code...
E:/Exp 19.pl compiled, 6 lines read - 836 bytes written, 8 ms

yes
| ?- student(rahul,Subject,Code).

Code = cs501
Subject = ai
```

Result

Thus, the Student-Teacher-Subject database was successfully implemented.

20. Prolog Program for Planets Database

Aim

To create a Prolog database containing information about planets.

Algorithm

1. Define planet facts.
2. Store planet name and properties.
3. Query the database.
4. Match the requested planet.
5. Retrieve its properties.
6. Display the result.

Program

```
planet(mercury,   rocky).
planet(venus,     rocky).
planet(earth,     rocky).
planet(mars,      rocky).
planet(jupiter,       gas).
planet(saturn, gas).
planet(uranus, gas).
planet(neptune, gas).
```

Query

?- planet(earth, Type).

Output

```
yes
| ?- consult('E:/Exp 20.pl').
compiling E:/Exp 20.pl for byte code...
E:/Exp 20.pl compiled, 7 lines read - 659 bytes written, 8 ms

yes
| ?- planet(earth).

yes
| ?- planet(X).

X = mercury ? |
```

Result

Thus, the Planet database was successfully created.

21. Prolog Program for Towers of Hanoi

Aim

To implement the Towers of Hanoi problem using Prolog recursion.

Algorithm

1. If there is one disk, move it directly.
2. Move N-1 disks from source to auxiliary.
3. Move the largest disk to destination.
4. Move N-1 disks from auxiliary to destination.
5. Repeat recursively.
6. Print all moves.

Program

```
hanoi(1, A, B, _) :-
   write('Move disk from '),
   write(A),
   write(' to '),
   write(B), nl.

hanoi(N, A, B, C) :-
   N > 1,
   N1 is N - 1,
   hanoi(N1, A, C, B),
   hanoi(1, A, B, C),
   hanoi(N1, C, B, A).
```

Query

?- hanoi(3, left, right, middle).

Output

```
(16 ms) yes
| ?- hanoi(3, left, right, middle).
Move disk from left to right
Move disk from left to middle
Move disk from right to middle
Move disk from left to right
Move disk from middle to left
Move disk from middle to right
Move disk from left to right

true ?
```

Result

Thus, Towers of Hanoi was successfully implemented using recursion.

22. Prolog Program to Check Whether a Bird Can Fly

Aim

To write a Prolog program to determine whether a particular bird can fly.

Algorithm

1. Store bird facts.
2. Define birds that normally fly.
3. Define exceptions.
4. Create the can_fly rule.
5. Query a bird.
6. Display whether it can fly.

Program

```
bird(sparrow).
bird(eagle).
bird(penguin).
bird(ostrich).

cannot_fly(penguin).
cannot_fly(ostrich).

can_fly(Bird) :-
bird(Bird),
\+ cannot_fly(Bird).
```

Queries

```
?- can_fly(sparrow).
```

Output:

```
compiling E:/AI EXP 22.pl for byte code...
E:/AI EXP 22.pl compiled, 12 lines read - 896 bytes written, 7 ms

yes
| ?- can_fly(penguin).

no
| ?- cannot_fly(penguin).

yes
| ?- |
```

Result

Thus, the program successfully determines whether a given bird can fly.

23. Prolog Program for Family Tree

Aim

To implement a family tree using Prolog facts and rules.

Algorithm

1. Define parent facts.
2. Define male and female facts.
3. Define father and mother rules.
4. Define sibling relationship.
5. Define grandparent relationship.
6. Query family relationships.

Program

```
male(john).
male(mike).
female(sarah).
female(anna).

parent(john, mike).
parent(sarah, mike).
parent(mike, anna).

father(X, Y) :-
male(X),
parent(X, Y).

mother(X,  Y)  :-
female(X),
parent(X, Y).

grandparent(X, Y) :-
   parent(X,  Z),
   parent(Z, Y).
```

Query

```
?- grandparent(john, anna).
```

Output

```
| ?- consult('E:/AI EXP 23.pl').
compiling E:/AI EXP 23.pl for byte code...
E:/AI EXP 23.pl compiled, 20 lines read - 1709 bytes written, 8 ms

(31 ms) yes
| ?- grandparent(john, anna).

yes
```

Result

Thus, the family tree was successfully implemented.

24. Prolog Program for Diet Suggestion Based on Disease

Aim

To create a simple Prolog rule-based system that demonstrates food-category suggestions based on a stated condition.

Algorithm

1. Define disease facts.
2. Define general food-category rules.
3. Match the disease with a rule.
4. Return the corresponding general suggestion. 5.
   Query the disease.
6. Display the result.

Program

```
disease(fever).
disease(cold).
disease(anemia).

diet(fever, 'fluids and light foods').
diet(cold, 'warm fluids and balanced foods').
diet(anemia, 'iron-rich foods').

suggest(Disease, Diet) :-
    disease(Disease),
    diet(Disease, Diet).
```

Query

```
?- suggest(fever, Diet).
```

Output

```
  ?- consult('E:/AI EXP 24.pl').
:ompiling E:/AI EXP 24.pl for byte code...
::/AI EXP 24.pl compiled, 11 lines read - 1097 bytes written, 9 ms

res
  ?- suggest(fever, Diet).

)iet = 'fluids and light foods'

res
  ?- |
```

Result

Thus, a simple rule-based diet suggestion system was implemented successfully.

25. Prolog Program for Monkey Banana Problem

Aim

To implement the Monkey Banana problem using Prolog state-space reasoning.

Algorithm

1. Represent monkey, box, banana, and position as a state.
2. Define movement of monkey.
3. Define pushing the box.
4. Define climbing onto the box.
5. Define taking the banana.
6. Search for a sequence that reaches the banana.

Program

```
move(state(P, P, onfloor, B),
state(Q, Q, onfloor, B)) :-
   P \= Q.

push(state(P, P, onfloor, P),
    state(Q, Q, onfloor, Q)) :-
   P \= Q.

climb(state(P, P, onfloor, B),
       state(P, P, onbox, B)).

 take(state(P, P, onbox, P),
    state(P, P, onbox, taken)).

solve(State, State) :-
   State = state(_, _, _, taken).

solve(State, Final) :-
   move(State, New),
   solve(New, Final).

solve(State, Final) :-
push(State,       New),
solve(New, Final).

solve(State, Final) :-
   climb(State, New),
   solve(New, Final).

solve(State, Final) :-
   take(State, New),
   solve(New, Final).
```

Query

?- solve(state(a,a,onfloor,a), Final).

Output

```
| ?- consult('E:/AI EXP 25.pl').
compiling E:/AI EXP 25.pl for byte code...
E:/AI EXP 25.pl compiled, 32 lines read - 2842 bytes written, 10 ms

yes
| ?- solve(state(a,a,onfloor,a), Final).

Final = state(a,a,onbox,taken) ?
```

Result

Thus, the Monkey Banana problem was implemented using state transitions and logical rules.

26. Prolog Program for Fruit and Color Using Backtracking

Aim

To implement a Prolog database for fruits and their colors and demonstrate backtracking.

Algorithm

1. Define fruit-color facts.
2. Query a fruit to obtain its color.
3. Query a color to find matching fruits.
4. Use ; to request another solution.
5. Prolog backtracks through the facts.
6. Display all matching solutions.

Program

```
fruit_color(apple, red).
fruit_color(banana, yellow).
fruit_color(orange, orange).
fruit_color(grapes, purple).
fruit_color(watermelon, green).
fruit_color(cherry, red).
```

Query

```
?- fruit_color(Fruit, red).
```

Output

```
| ?- consult('E:/AI EXP 26.pl').
compiling E:/AI EXP 26.pl for byte code...
E:/AI EXP 26.pl compiled, 6 lines read - 750 bytes written, 8 ms

yes
| ?- fruit_color(Fruit, red).

Fruit = apple ?
```

Result

Thus, fruit-color information was successfully retrieved using Prolog backtracking.

27. Prolog Program for Best First Search

Aim

To implement Best First Search using heuristic values.

Algorithm

1. Define the graph.
2. Assign heuristic values to nodes.
3. Start from the initial node.
4. Select the node having the smallest heuristic value.
5. Expand its neighbors.
6. Continue until the goal is reached.

Program

```
edge(a, b).
edge(a, c).
edge(b, d).
edge(c, d).
edge(d, e).

h(a, 5).
h(b, 3).
h(c, 2).
h(d, 1).
h(e, 0).

best_first(Start, Goal, Path) :-
   search([Start], Goal, [], RevPath),
   reverse(RevPath, Path).

search([Goal|_], Goal, Visited, [Goal|Visited]).

search([Node|_], Goal, Visited, Path) :-
   findall(H-Next,
      (edge(Node, Next),
       \+ member(Next, Visited),
       h(Next, H)),
      Children),
   sort(Children, Sorted),
   Sorted = [_-Next|_],
   search([Next], Goal, [Node|Visited], Path).
```

Query

```
?- best_first(a, e, Path).
```

Output

```
| ?- consult('E:/AI EXP 27.pl').
compiling E:/AI EXP 27.pl for byte code...
E:/AI EXP 27.pl compiled, 27 lines read - 2898 bytes written, 17 ms

yes
| ?- best_first(a, e, Path).

Path = [a,c,d,e] ? |
```

Result

Thus, Best First Search was successfully implemented using heuristic values.

28. Prolog Program for Medical Diagnosis

Aim

To implement a simple rule-based medical diagnosis demonstration using symptoms.

Algorithm

1. Define observed symptom facts.
2. Define rules connecting symptom combinations to diagnosis categories.
3. Query the diagnosis.
4. Prolog checks the symptoms.
5. Match the appropriate rule.
6. Display the rule-based result.

Program

```
symptom(john, fever).
symptom(john, cough).
symptom(sarah, headache).
symptom(sarah, fever).
diagnosis(Person, flu_like_condition) :-
    symptom(Person, fever),
    symptom(Person, cough).

diagnosis(Person, fever_condition) :-
    symptom(Person, fever).

diagnosis(Person, headache_condition) :-
    symptom(Person, headache).
```

Query

```
?- diagnosis(john, D).
```

Output

```
| ?- consult('E:/AI EXP 28.pl').
compiling E:/AI EXP 28.pl for byte code...
E:/AI EXP 28.pl compiled, 15 lines read - 1209 bytes written, 8 ms

yes
| ?- diagnosis(john, D).

D = flu_like_condition ? |
```

Result

Thus, a simple rule-based medical diagnosis demonstration was successfully implemented.

29. Prolog Program for Forward Chaining

Aim

To implement forward chaining using facts and rules.

Algorithm

1. Store initial facts.
2. Store logical rules.
3. Check whether rule conditions are satisfied.
4. Add the conclusion as a new fact.
5. Repeat until no new facts can be inferred.
6. Query the final knowledge base.

Program

```
fact(sunny).
fact(hot).
rule(pleasant, [sunny, hot]).

forward :-
    rule(New, Conditions),
    all_true(Conditions),
    \+ fact(New),
    assertz(fact(New)),
    forward.

forward.

all_true([]).

all_true([H|T]) :-
    fact(H),
    all_true(T).
```

Query

```
?- forward.
true.
?- fact(pleasant).
true.
```

Output

```
| ?- consult('E:/AI EXP 29.pl').
compiling E:/AI EXP 29.pl for byte code...
E:/AI EXP 29.pl compiled, 19 lines read - 1386 bytes written, 8 ms

yes
| ?- fact(pleasant).

no
| ?- true.

yes
```

Result

Thus, forward chaining was successfully implemented by deriving new facts from existing facts and rules.

30. Prolog Program for Backward Chaining

Aim

To implement backward chaining using goals, facts, and rules.

Algorithm

1. Start with a query or goal.
2. Check whether the goal is a known fact.
3. If not, find a rule whose conclusion matches the goal.
4. Treat the rule conditions as sub-goals.
5. Recursively prove every sub-goal.
6. Return true when all required conditions are satisfied.

Program

```
fact(sunny).
fact(hot).
rule(pleasant, sunny).
rule(pleasant, hot).
prove(X) :-
    fact(X).

prove(X) :-
    rule(X, Y),
    prove(Y).
```

Query

```
?- prove(sunny).
```

Output:

```
| ?- consult('E:/AI EXP 30.pl').
compiling E:/AI EXP 30.pl for byte code...
E:/AI EXP 30.pl compiled, 12 lines read - 804 bytes written, 8 ms

(16 ms) yes
| ?- prove(sunny).

true ? |
```

Result

Thus, backward chaining was successfully implemented by starting from the goal and recursively proving its supporting conditions.

31. Web Blog Using WordPress – Anchor Tag and Title Tag

Aim

To create a simple web blog demonstrating ***Title Tag, Heading Tag, Paragraph Tag, Anchor Tag, Image Tag, and List Tag***.

Algorithm

1. Create a basic HTML webpage structure.
2. Add a title using the <title> tag.
3. Add blog headings using <h1> and <h2>.
4. Add text using <p> and links using <a>.
5. Add an image and list of blog topics.
6. Open the HTML page in a browser and verify all elements.

Program

```
<!DOCTYPE  html>
<html>
<head>
   <title>My AI Technology Blog</title>
</head>

<body>

   <h1>Welcome to My AI Technology Blog</h1>

   <h2>Artificial Intelligence</h2>

   <p>
      Artificial Intelligence enables computers to perform
      tasks that normally require human intelligence.
   </p>

   <h2>Useful Links</h2>

   <a href="https://www.google.com">
      Visit Google
   </a>

   <br><br>

   <a href="https://www.wikipedia.org">
      Visit Wikipedia
   </a>

   <h2>Blog Topics</h2>
```

```
    <ul>
        <li>Artificial Intelligence</li>
        <li>Machine Learning</li>
        <li>Deep Learning</li>
        <li>Robotics</li>
    </ul>

    <h2>About the Blog</h2>

    <p>
        This blog provides basic information about
        modern AI and technology.
    </p>

</body>
</html>
```

Result

The web blog was successfully created with ***Title, Heading, Paragraph, Anchor, and List tags***. The hyperlinks can be clicked to navigate to the specified websites.